IntraSpanNet: Identifying Security Appliances, Endpoint Agents, and Common Software from Five-Packet Intranet Flows without Active Scanning

Chenhe Li, Independent Researcher

Abstract

We present IntraSpanNet, a byte-level model that runs beside an intranet traffic probe. From the first five packets of a flow it returns the product that produced the flow, or an explicit unknown [1], [7]. The label set covers 18 items across three tiers: security appliances, endpoint security agents, and common software. The model masks the IPv4 addresses, packs the packets into a fixed 1,600-byte vector, and encodes it with four Gated DeltaNet blocks whose 256-dimensional CLS state feeds four task heads and a prototype gate [1], [2], [4], [11]. On a 503-flow test split the four-arm ensemble reaches 0.8787 product accuracy and 0.9245 tier accuracy, with 0.9784 gate AUROC, and rejects all 18 unknown-product bins at a threshold calibrated on the development half [1], [9], [11]. The release includes a 42 KB C probe, a CLI, an HTTP service, an enrollment routine that adds a product from three sub-prototypes, and fine-tuning and reinforcement-learning hooks.

Index Terms: passive asset discovery, intranet traffic analysis, traffic probe, byte-level classification, tokenizer-free learning, Gated DeltaNet, open-set recognition, encrypted traffic analysis

I Introduction

Endpoint agents are installed by operations, VPN endpoints and the web application firewall by the network team, and the NDR sensor by whoever owns monitoring, so no single owner holds the consolidated product view for a host [17], [19], [21]. Asset records are populated at provisioning time and reconciled rarely afterwards, so a host that has been reimaged, reassigned, or fitted with a new agent can carry a product set that differs from the recorded entry.

Port and protocol rules do not close that gap. The label set used here is closed at 18 products and services, listed in Appendix A with their type and tier, and it contains three VPN products (wireguard, openvpn, and stunnel) that accept operator-configured ports [19], three web-facing products (squid_proxy, ntopng, and guacamole_mgmt) that share 80 and 443 in the default configuration [19], and four endpoint agents (windows_defender, clamav_freshclam, velociraptor, and osquery) whose update traffic follows the same HTTPS pattern [20], [21]. Rule-based fingerprinting resolves the operating system from header quirks [17], the protocol from payload patterns [19], or the TLS stack from a handshake hash [18], whereas learned device identification resolves the device type from flow statistics [20], [21]. Neither produces a product label from a closed set, and neither supplies an abstain option calibrated on held-out sessions [11], [12]. A probe sidesteps the scanning problem entirely, because its span port already carries the traffic of every host that matters and nothing has to be sent for the inventory to grow.

IntraSpanNet addresses that setting with a compact byte-level model. The input is the first five packets of a flow, packed into 1,600 bytes with the IPv4 addresses zeroed [1], [7]. The encoder is four Gated DeltaNet blocks of $d_{model} = 256$ and $d_{state} = 16$, giving 2.86M parameters per arm [2], [3], [4]. The classifier is a four-arm ensemble of 11.44M parameters that occupies 46 MB on disk [1], [8], [9]. A separate prototype gate decides whether the product head may be trusted [11], [12], [13], so the system returns a product label, a type, a tier, a prototype distance, and a confidence, or an explicit unknown.

For a flow f, with P the closed set of 18 products listed in Appendix A, T the set of product types, and G the set of tiers, the classifier returns a tuple (┼╥, t╟é, ─¥, d, s) with ┼╥ in P or unknown, t╟é in T, ─¥ in G, a prototype distance d, and a confidence s [11], [13]. Rejection is a threshold of ╧ä = 0.1041, the 68th percentile of development distances, applied to the distance between the flow embedding and the nearest of three class prototypes [12], [13]. It is not an additional output class, so no unknown-class examples are required during training, which matters because the set of products absent from a given network cannot be enumerated in advance [5], [6].

On the 40,283 labelled flows used here the ensemble reaches 0.8787 product accuracy on a 503-flow test half, with 0.9245 tier accuracy and 0.9784 gate AUROC, and it rejects all 18 unknown-product bins in the acceptance set [14], [16]. The probe is a single C file of 42 KB and peak VRAM is 3.40 GB at batch 128 on one RTX 4090 [28], [29], so the inference path fits on the card that also runs the analyst's tooling. The same property makes the result sensitive, because an adversary who can enumerate the defensive stack of a host can select techniques that avoid it, and Section VI-E states the release constraints that follow [14], [22].

This paper makes four contributions.

- We introduce IntraSpanNet, a byte-level classifier that assigns flows to 18 security products and an explicit unknown class from the first five packets, using 2.86M parameters per arm and 11.44M for the four-arm ensemble, with no tokenizer and no externally pre-trained corpus [1], [6], [9].
- We describe a rejection mechanism based on three prototypes per class and a threshold calibrated on held-out sessions, which rejects all 18 unknown-product bins in the acceptance set without training on unknown samples [11], [12].
- We describe the path around the classifier, namely a 42 KB C probe that emits the input vector at the fifth packet, a CLI, an HTTP service, and an enrollment routine that adds a new product from three sub-prototypes without retraining [11], [25].
- We release an inference stack with checksums that reproduces the reported numbers without redistributing any capture [28], [29].

II Related Works

II-A Traffic fingerprinting and asset discovery

p0f infers an operating system from TCP/IP header characteristics such as the initial TTL and the ordering of TCP options, using a hand-written signature table and no training data [17]; nDPI classifies protocols by matching payload and header patterns at line rate [19]; and JA3 with JA3S hash the fields of the TLS ClientHello and ServerHello into short strings that identify client and server implementations [18]. Learned device identification moves one step further: IoT Sentinel trains a random forest over header features to name device types across a small device vocabulary [21], and Sivanathan et al. separate smart-home and campus device classes on flow statistics alone [20]. All of these stop at the operating system, the protocol, or the device type, and none produces a product label from the closed set used here or supplies an abstain option.

II-B Byte-level traffic classification

ET-BERT tokenizes datagrams before pre-training a Transformer with masked language modeling [7], YaTC renders flows as images and pre-trains a masked autoencoder over patches [8], and NetMamba applies a unidirectional Mamba backbone to packet sequences [9]; all three require a pre-training stage over an external corpus, and all three descend from the byte-level and subword conventions established for text [5], [6]. MambaNetBurst removes both the tokenizer and the pre-training stage and learns directly from raw bytes with a compact Mamba-2 stack of 2.7M parameters [1], which is the closest architectural relative of the present work. IntraSpanNet keeps the byte vocabulary, the CLS readout, and the fixed-length burst convention of that design and replaces the mixer with four

Gated DeltaNet blocks [4], which retain the linear cost of the state-space formulation [2], [3] while adding a delta-rule update to the transition.

II-C Foundation models and open-set recognition

netFound learns packet representations from a large token corpus and reports frozen-encoder results alongside explicit corrections for dataset shortcuts, at roughly two orders of magnitude more parameters than one 2.86M-parameter IntraSpanNet arm [10]. Open-set recognition treats rejection as a decision separate from classification, since softmax confidence is a poor boundary estimator [13] and distance to class prototypes is a calibrated alternative [11], [12]. We keep three sub-prototypes per class inside 54 prototype rows per gate arm, define distance as one minus the maximum cosine similarity, and calibrate ≐ä on the development half only.

II-D Evaluation practice

Evaluation practice in this area has been criticised on two counts that matter here, since packet-level splits can leak sequence numbers into the test set [15] and reported gains sometimes reflect shortcut learning rather than the task [14], which is why a common protocol has been proposed for comparison [16]; this paper therefore fixes one protocol, scores the test half once after every choice has been made on the development half, and releases the evaluation script together with the checkpoints.

TABLE I. Placement of IntraSpanNet among fingerprinting, learned classification, and foundation-model approaches.

Method	Representation	Tokenizer	Pre-training	Sequence model	Target
p0f [17]	TCP/IP header fields	none	none	signature table	OS
nDPI [19]	payload + header	none	none	DPI rules	protocol
JA3/JA3S [18]	TLS hello fields	none	none	hash	client/server
IoT Sentinel [21]	header features	none	none	random forest	device type
ET-BERT [7]	datagram token	BERT vocab	masked LM	Transformer	application
YaTC [8]	flow image patch	patch	masked AE	Transformer	application
NetMamba [9]	packet token	vocab	self-supervised	Mamba	application
MambaNetBurst [1]	raw byte	none	none	Mamba-2	application
netFound [10]	packet token	vocab	large corpus	Transformer	multiple
IntraSpanNet	raw byte	none	contrastive, in-domain	GDN	security product

III Architecture

III-A Input representation

Each flow is reduced to a fixed vector of 1,600 bytes: five packets of up to 320 bytes, of which the first 80 are headers and the next 240 are payload, with bytes 12 through 19 holding the IPv4 addresses zeroed before the vector leaves the probe while ports, protocol numbers, flags, and payload content are preserved [1], [7].

$$x = [\, b_1, b_2, \Gamma\text{Ç}^{a}, b_{1600} \,], \; b_i \Gamma\hat{e}\hat{e} \{0, \Gamma\text{Ç}^{a}, 255\} \tag{1}$$

Shorter flows are zero-padded and longer flows truncated at the same 1,600-byte boundary, Ethernet, VLAN, and raw IPv4 framing are all accepted, and parsing never sits on the hot path because the probe emits the vector as soon as the fifth packet of a flow arrives [28]. The five-packet budget carries the transport handshake and the first application message, which is the segment in which the 18 products of Appendix A disclose a version string, a certificate, or a fixed update header, whereas an address is a property of a site rather than of a product [17], [19], [21].

III-B Byte embedding, backbone, and heads

Each byte is mapped to a 256-dimensional embedding, a learnable CLS token is appended, positional information is added, and four GDN blocks process the sequence, whose final CLS state feeds four heads while a training-only domain head loads with strict=False and is discarded at inference [1], [4]. The four heads together hold about 0.4M of the 2.86M parameters, so the encoder accounts for 86% of the model, and the malicious head is trained on public malware traffic and reported separately because that corpus does not match the collection conditions of any particular site.

TABLE II. Backbone configuration of one arm.

Component	Setting
Byte embedding	256
GDN blocks	4, d_{model} = 256, d_{state} = 16
CLS token	learnable, appended after the last byte
Feature	final CLS state, 256 dims
State dict entries	103, of which 52 are GDN block entries
Parameters per arm	2.86M

TABLE III. Task heads, about 0.4M parameters in total.

Head	Layers	Outputs	In main results
Product	256-512-18	18	yes
Tier	256-3 linear	3	yes
Type	256-512-10	10	yes
Malicious	256-512-11	11	reported separately

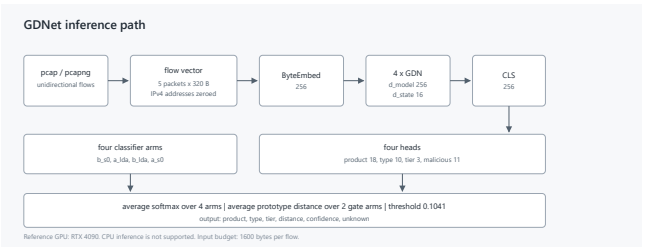


Fig. 1. Inference path of one arm.

III-C Open-set decision

Three prototypes per class are produced by KMeans over the training embeddings, giving 54 prototype rows per gate arm, and for a feature f and the prototype set P_c of class c the distance and the decision rule are

$$d(f) = 1 \; \Gamma\hat{e}\text{Æ} \; \max_{c,k} \cos(f, p_{c,k}), \; p_{c,k} \; \Gamma\hat{e}\hat{e} \; P_c \tag{2}$$

$$\text{┼╖} = \operatorname{argmax}_c s_c(f) \; \text{if } d \text{╠ä}(f) \; \Gamma\ddot{e}\tilde{n} \; \text{≐ä}, \; \text{otherwise unknown} \tag{3}$$

where s is the averaged softmax vector, d ╠ä the distance averaged over the two gate arms, and ≐ä = 0.1041 the 68th percentile of development distances [11], [12], [13]. The cosine distance is bounded by the range [0, 2], so the threshold is a quantile of the development distribution rather than an absolute similarity, and any new site requires its own recalibration because the threshold is a property of the traffic rather than of the model [12], [13].

III-D Ensemble, complexity, and footprint

Six arms were trained, formed by crossing two pre-training runs A and B with three training variants: a standard multi-task objective (S0), logit adjustment by class prior (LDA), and hard-negative weighting inside the contrastive term (HN). Four were retained, A-S0, A-LDA, B-LDA and B-S0, while A-HN and B-HN reached a higher AUROC yet their gate distance moved whenever the threshold was adjusted, so they were dropped on stability grounds rather than on accuracy [1], [9]; averaging is unweighted.

TABLE IV. The six trained arms. Released checkpoints carry the same labels in lower case, for example stage3_a_s0.pth for A-S0.

Arm	Run	Training variant	Role in release
A-S0	A	standard multi-task	classifier
A-LDA	A	logit adjustment	classifier and gate
B-LDA	B	logit adjustment	classifier and gate
B-S0	B	standard multi-task	classifier
A-HN	A	hard-negative weighting	dropped
B-HN	B	hard-negative weighting	dropped

$$s(f) = (1/4) \sum_{j=1..4} softmax(z_j(f)),\ d(f) = (1/2) \sum_{j \in \{A\text{-}LDA,\ B\text{-}LDA\}} d_j(f) \quad (4)$$

A Gated DeltaNet block is linear in sequence length, so a forward pass costs $O(L \cdot d_{model} \cdot d_{state})$ with L = 1,600 [2], [3], [4], and memory is dominated by activations rather than weights, with 46 MB of parameters against 3.40 GB of peak VRAM at batch 128.

```
Algorithm 1: Inference for a single flow
Input: packet stream, threshold τ
       arms S = {A-S0, A-LDA, B-LDA, B-S0}
Output: product, type, tier, distance, confidence, or
unknown

 1  collect the first five packets of a unidirectional flow
 2  x <- 1600-byte vector with bytes 12:20 zeroed
 3  for each arm j in S:
 4      e_j <- GDN_encoder_j(x)                  # 256-d CLS
 5      s_j <- softmax(product_head_j(e_j))      # 18 logits
 6  s <- mean over j of s_j
 7  d <- mean over gate arms of 1 - max cosine(e, prototype)
 8  if d <= τ: return argmax s, type head, tier head, d,
max s
 9  else:      return unknown, d
```

IV Experiments

IV-A Data, splits, and implementation

The labelled set contains 40,283 flows over 18 classes captured on laboratory hosts and virtual machines, and each product was exercised through its normal update, telemetry, or management path rather than through synthetic payloads, because a security product's traffic is dominated by the way it contacts its own infrastructure [7], [8], [30], [32]. The split is at flow level rather than session level, and because each class appears in exactly one held-out session the test figures describe those sessions rather than a new organisation; 14 sessions cover 18 classes because a single capture session exercises several products on one host [14], [16].

TABLE V. Data, splits, and training configuration.

Item	Value
Classes	18 products and services
Labelled flows	40,283
Capture sources	laboratory hosts and virtual machines
Validation flows	1,006, from 14 held-out sessions
Dev / test split	503 / 503 at flow level
Selection	threshold, checkpoint, and ensemble on dev only
Test usage	once
Hardware	4 × RTX 4090, fp32
Stage 1	contrastive, NT-Xent, 33,184 training-only flows
Stage 2	supervised fine-tuning, 60 epochs, multi-task
Stage 3	REINFORCE, 300 epochs
Reference stack	Python 3.12, PyTorch 2.14

The training code is not in the release because it assumes the laboratory data layout of 40,283 flows and would require cleanup before another group could execute it, whereas the released checkpoints, 54 prototype rows, threshold of 0.1041, and evaluation script reproduce the test-half numbers directly [28], [29]. Model selection is confined to the 503-flow development half, so the 503-flow test half is scored once [14], [16].

Training follows three stages: the contrastive stage optimises NT-Xent over 33,184 training-only flows [23], fine-tuning adds a domain-adversarial term [24] under AdamW [26] with defaults inherited from Adam [27], and the last stage applies REINFORCE to three rewards, namely correct product, correct type, and honest rejection [25].

Reproducibility has two halves: the released checkpoints, 54 prototype rows, threshold and evaluation script reproduce the test-half figures whenever the split is available, while the split itself is withheld because it derives from laboratory traffic; the probe-to-service path collected 99,888 flows in 30 seconds on the test host and the service labelled container-management traffic as unknown, which is the expected behaviour outside the product domain.

IV-B Metrics

Product accuracy counts exact product labels and tier accuracy counts exact tier labels, macro-F1 averages over the classes that have at least one test flow according to

$$F1_{macro} = (1/|C^+|) \sum_{c \in C^+} 2 P_c R_c / (P_c + R_c) \quad (5)$$

where C^+ denotes that set, gate AUROC compares known flows against unknown and novel flows [12], [13], and unknown rejection reports the share of unknown-product bins correctly refused, while malicious macro-F1 is computed on the deheld-out USTC-TFC2016 test split of 1,242 flows [32] and accuracy intervals are Wilson intervals at 95%.

V Results

TABLE VI. Closed-set accuracy, gate quality, and rejection.

Metric	Test (503 flows)	Wilson ±	All validation (1006 flows)	Wilson ±
Product accuracy	0.8787	0.029	0.8817	0.020
Tier accuracy	0.9245	0.023	0.9334	0.016
Macro-F1, supported classes	0.7621	—	0.8240	—
Gate AUROC	0.9784	—	0.9820 (dev)	—
Unknown rejection	18/18	—	18/18	—
Malicious macro-F1, dedup USTC	—	—	0.9702	—
Malicious binary F1	—	—	>0.994	—

V-A Security product identification

The product head attains 0.8787 on the test half and 0.8817 across the full validation set, tier accuracy 0.9245 and 0.9334, and gate AUROC 0.9784 and 0.9820 [1], [9]. All 18 unknown-product bins are rejected at the fixed threshold of 0.1041, with the rejection rate unchanged between q = 0.64 and q = 0.75 [11], [12]; wireguard has a single test flow, so its class score moves between 1.0 and 0.0 on one error, and the test-half macro-F1 of 0.7621 is therefore not a headline figure [14], [16]. The all-validation value of 0.8240 is the stable reference, and the malicious head reaches 0.9702 macro-F1 on 1,242 USTC-TFC2016 test flows without falling below 0.994 binary F1 [15], [32].

V-B Acceptance and enrollment

TABLE VII. Acceptance and enrollment on 264 product bins and 18 unknown bins.

Item	Result
Product bins	264
Unknown bins	18, all rejected
Products correct, raw ensemble	8/10
Products correct, with timing rule	9/10, rule not released
stunnel recall	0/30 at every threshold
wireguard	2 bins, treated as noise
HTTP service, FLW1 records	6/6
Enrollment, smb	8/8 after three sub-prototypes

A timing rule over inter-arrival times raises the acceptance figure from 8 to 9 of 10 and is excluded from the release because it is a second model without an independent calibration, while stunnel produces 0 hits from 30 bins at every threshold examined; enrollment

on smb reaches 8 of 8 after three sub-prototypes were registered, which shows the mechanism functions but does not supply the data stunnel lacks [11], [21]. The HTTP service returns 6 of 6 known flows on FLW1 records, which is the only end-to-end check that traverses the code path a deployment would use [28], [29].

V-C Throughput and memory

TABLE VIII. Throughput and memory on one RTX 4090 in fp32, measured at batch 128 for VRAM and at batch 1 for latency.

Configuration	Throughput	Batch-1 latency	Peak VRAM, batch 128
Single arm	1,017 flows/s	10.7 ms	3.37 GB
Four-arm ensemble	253 flows/s	43.1 ms	3.40 GB
Distilled student	1,011 flows/s	10.6 ms	3.37 GB

The probe captures faster than the 253-flow/s ensemble classifies, so a deployment must sample, shard across several devices, or classify on demand, and an 8 GB card retains headroom at batch 128 with 3.40 GB of peak VRAM [28], [29]. The distilled student reaches 1,011 flows/s at 0.8231 validation accuracy, 5.6 points below the ensemble, and is kept experimental for that reason [23].

V-D Public split audit

Benchmark	Train	Test	Train present in test	Share of test	Clean test
ISCXVPN2016	13,281	1,384	1,106	79.9%	278
ISCXTor2016	11,655	1,458	0	0%	1,458
USTC-TFC2016	37,323	6,662	5,605	84.1%	1,057
CICIoT2022	18,734	1,951	1,571	80.5%	380
CrossPlatform-Android	44,700	4,656	3,733	80.2%	923
CrossPlatform-iOS	40,379	4,205	3,396	80.8%	809

TABLE IX. Exact byte-level train/test overlap in six public benchmarks.

Every flow was hashed twice, once over the PNG file bytes and once over the decoded grayscale pixels, and the two hashes agree on all 250,072 flows. A duplicate is an exact byte match at the 1,600-byte input unit, not a near match. Five of the six splits place about 80% of their test flows in the training set, which means a score computed on those splits measures recall of training data rather than generalisation [14], [15], [16]. ISCXTor2016 duplicates nothing and is the only split in the set whose test figure carries its usual meaning.

Label conflicts across splits are zero and every duplicate stays inside its class, which is consistent with independent sampling from one flow pool without cross-split deduplication. The finding is not universal: the Lens official splits carry 7.0% to 17.2% burst overlap on the four in-scope tasks and 0.05% to 0.5% on the two mobile tasks, with at most 57 conflicting rows against test.

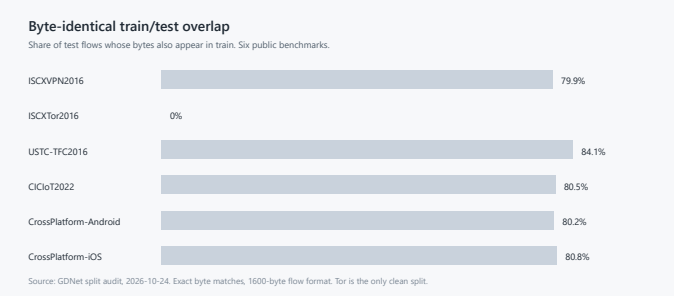


Fig. 2. Exact byte-level train/test overlap in the six audited benchmarks.

V-E Deduplicated evaluation

Item	VPN	USTC	CIoT
Clean test flows	278	1,057	380
Accuracy on clean set	0.9376	0.9981	0.9868
Macro-F1 on clean set	0.9418	0.9671	0.9823
Full-set macro-F1	0.9861	0.9949	0.9965
Drop	4.4 pp	2.8 pp	1.4 pp

TABLE X. Re-evaluation on the deduplicated subsets of three affected benchmarks.

Re-scoring on the clean subsets costs 4.4 points of macro-F1 on VPN, 2.8 on USTC and 1.4 on CIoT. The leak therefore inflated the reported scores without accounting for all of them, which is itself evidence that the model learned something beyond the duplicated flows. Both figures are reported for the three benchmarks that were rerun; the other three were not rerun and neither figure is claimed for them.

V-F Ablations and variants that did not ship

TABLE XI. Variants and ablations.

Variant	Result	Decision
Auxiliary pre-training (field, order, reconstruction)	Probe +9.35 points; downstream E1 below E0+LDA	not used
Timing rule	Acceptance 8 to 9; unstable at nearby thresholds	not released
Distillation	0.8231 validation accuracy; 5.6 points below ensemble	experimental
stunnel threshold tuning	0/30 at every threshold	needs data or enrollment
Two hard-negative arms	Higher AUROC; gate distance moved with ≟ä	dropped

Auxiliary pre-training is the one that looked most promising, and it raised the probe score by 9.35 points before losing to the plain E0-plus-LDA pipeline downstream, which is why it was dropped rather than tuned [6], [23]. Depth, width, state size, embedding dimension, and packet budget were fixed once from the deployment constraint and the MambaNetBurst reference rather than swept, so the available compute went to the 264-bin acceptance set and the end-to-end service check instead [1], [3].

V-G Baselines under a capture-level split

Two model-free references were trained on USTC-TFC2016, over 1,105 flows split by capture into 7 training and 4 test captures.

Features	Flow-level split (acc / F1)	Capture-level split (acc / F1)
256-bin byte histogram	1.0000 / 1.0000	0.7766 / 0.4998
Hashed byte 3-grams	not run	0.9619 / 0.7401

TABLE XIII. Linear baselines on USTC-TFC2016 under two split protocols.

Under a flow-level split the histogram pipeline reaches 1.0000 accuracy, which is the artefact described in Section V-D. Under the capture-level split it drops to 0.7766, while the 3-gram representation holds 0.9619, which indicates that local byte order rather than byte frequency carries most of the signal on this dataset. The difference between the two protocols, rather than either number alone, is the result: a linear model with no capacity to memorise reproduces the leakage effect on a third dataset.

VI Discussion

VI-A Five packets are enough because the handshake is where the product signs itself.

The acceptance set reaches 8 of 10 products from the first five packets alone, because each product in Appendix A emits a version string, a certificate, a fixed header, or an update request inside that budget [17], [19], [21].

VI-B Model size follows the deployment constraint.

Each arm holds 2.86M parameters and the ensemble 11.44M, which fits in 46 MB and reaches 3.40 GB of peak VRAM at batch 128, while the 42 KB probe runs in a container without a Python runtime [28], [29]; a larger model would plausibly score higher [1], [10], but the operating constraint here was the 8 GB card that also runs the analyst's tooling.

VI-C Rejection matters more than accuracy.

The gate rejects 18 of 18 unknown bins at ≐ä = 0.1041 with the operating point stable from q = 0.64 to q = 0.75 [11], [12], and without that rejection the 0.8787 product accuracy would send the responder toward software that is not installed whenever a flow falls outside the 18 classes [13], [22].

VI-D Limits of the evaluation.

The split is flow-level with one session per class, so the absolute figures describe those 14 sessions; the 503-flow test set yields a Wilson interval of ┬▒0.029 on product accuracy, and no interval is available for macro-F1 without the per-class counts; only one capture environment was used, so cross-site behaviour is unmeasured; the label set is closed at 18 products, so a product that updates silently will be rejected or misclassified; and no seed sweep was performed.

VI-E Use and misuse.

The release contains 19 weight files totalling 94.6 MB, no captures, and SHA-256 checksums, and it names defence products, so it is run only where authorisation exists and captures stay off public demonstrations [22]. The outputs are scores rather than verdicts, the malicious head is reported separately and excluded from automated response paths, and the input carries no addresses and no payload semantics beyond byte patterns.

VII Conclusion

IntraSpanNet names security products from the first five packets of a flow using 11.44M parameters and no tokenizer, and it declines to answer when a flow does not resemble anything in its training set, attaining on a 503-flow test split 0.8787 product accuracy, 0.9245 tier accuracy, and 0.9784 gate AUROC while rejecting all 18 unknown-product bins in the acceptance set [1], [9]. The parts that matter in deployment are the rejection path and the probe: the first keeps a wrong label from reaching a responder, and the second means the inventory is collected without touching a single host [11], [12], [22].

Acknowledgments

The Gated DeltaNet kernel is taken from flash-linear-attention [29], and the public benchmarks used for the malicious head were constructed by the groups that published them [30], [31], [32].

References

[1] G K Kulatilleke, S Layeghy, M Baktashmotlagh, and M Portmann, "MambaNetBurst: direct byte-level network traffic classification without tokenization or pretraining," arXiv:2605.11034, 2026; [2] A Gu and T Dao, "Mamba: linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023; [3] T Dao and A Gu, "Transformers are SSMs: generalized models and efficient algorithms through structured state space duality," in Proc ICML, 2024; [4] S Yang, J Kautz, and A Hatamizadeh, "Gated delta networks: improving Mamba2 with the delta rule," in Proc ICLR, 2025; [5] A Vaswani et al, "Attention is all you need," in Proc NeurIPS, 2017; [6] L Xue et al, "ByT5: towards a token-free future with pre-trained byte-to-byte models," TACL, vol 10, 2022; [7] X Lin, G Xiong, G Gou, Z Li, J Shi, and J Yu, "ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification," in Proc WWW, 2022; [8] R Zhao, M Zhan, X Deng, Y Wang, Y Wang, G Gui, and Z Xue, "Yet another traffic classifier (YaTC): a masked autoencoder for flow images," arXiv:2305.19077, 2023; [9] NetMamba: efficient network traffic classification via pre-training unidirectional Mamba, arXiv:2405.11886, 2024; [10] netFound: foundation model for network security, arXiv:2310.17025, 2023, revised 2026; [11] J Snell, K Swersky, and R Zemel, "Prototypical networks for few-shot learning," in Proc NeurIPS, 2017; [12] W J Scheirer, A de Rezende Rocha, A Sapkota, and T E Boult, "Toward open set recognition," IEEE TPAMI, vol 35, no 7, 2013; [13] A Bendale and T E Boult, "Towards open set deep networks," in Proc IEEE CVPR, 2016; [14] N Jacobs et al, "AI/ML for network security: the emperor has no clothes," in Proc ACM CCS, 2022; [15] R Zhao et al, "The sweet danger of sugar: the pitfalls of pcap-based encoders," in Proc ACM SIGCOMM, 2025, arXiv:2507.16438; [16] NetBench: a benchmark for network traffic classification, in Proc ACM FMSys, 2024; [17] M Zalewski, "p0f v3," https://lcamtufcoredumpcx/p0f3/; [18] J Althouse, "TLS fingerprinting with JA3 and JA3S," Salesforce Engineering, 2017; [19] L Deri, M Martinelli, T Bujlow, and A Cardigliano, "nDPI: open-source high-speed deep packet inspection," in Proc IEEE IWCMC, 2014; [20] A Sivanathan, H H Gharakheili, F Loi, A Radford, C Wijenayake, A Vishwanath, and V Sivaraman, "Characterizing and classifying IoT traffic in smart cities and campuses," in Proc IEEE INFOCOM Workshops, 2017; [21] M Miettinen, S Marchal, I Hafeez, N Asokan, A-R Sadeghi, and S Tarkoma, "IoT Sentinel: automated device-type identification for security enforcement in IoT," in Proc IEEE ICDCS, 2017; [22] R Sommer and V Paxson, "Outside the closed world: on using machine learning for network intrusion detection," in Proc IEEE S&P, 2010; [23] T Chen, S Kornblith, M Norouzi, and G Hinton, "A simple framework for contrastive learning of visual representations," in Proc ICML, 2020; [24] Y Ganin et al, "Domain-adversarial training of neural networks," JMLR, vol 17, no 59, 2016; [25] R J Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol 8, 1992; [26] I Loshchilov and F Hutter, "Decoupled weight decay regularization," in Proc ICLR, 2019; [27] D P Kingma and J Ba, "Adam: a method for stochastic optimization," in Proc ICLR, 2015; [28] A Paszke et al, "PyTorch: an imperative style, high-performance deep learning library," in Proc NeurIPS, 2019; [29] flash-linear-attention, https://githubcom/fla-org/flash-linear-attention; [30] G Draper-Gil, A H Lashkari, M S I Mamun, and A A Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in Proc ICISSP, 2016; [31] A H Lashkari, G Draper-Gil, M S I Mamun, and A A Ghorbani, "Characterization of Tor traffic using time-based features," in Proc ICISSP, 2017; [32] W Wang, M Zhu, J Wang, X Zeng, and Z Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in Proc IEEE ISI, 2017.

Appendix A: Label Set

TABLE XII. The 18 products and services used as the closed label set, with type and tier.

Product	Type	Tier
windows_defender	Antivirus	Security software
clamav_freshclam_win	Antivirus	Security software
clamav_freshclam	Antivirus	Security software
velociraptor	EDR / HIDS	Security software
velociraptor_win	EDR / HIDS	Security software
osquery	EDR / HIDS	Security software
wazuh_agent	EDR / HIDS	Security software
wireguard	Zero trust / VPN	Security appliance
openvpn	Zero trust / VPN	Security appliance
stunnel	Zero trust / VPN	Security appliance
squid_proxy	Internet behavior management	Security appliance
ntopng	NDR	Security appliance
guacamole_mgmt	Bastion host	Security appliance
waf_modsecurity	WAF	Security appliance
suricata_update	NDR	Security appliance
web_http	Browser	Common software
ssh	Remote administration	Common software
ftp	File transfer	Common software

No capture is redistributed in the release: the weights were trained on laboratory traffic and on the training splits of public benchmarks

[30], [31], [32], and the SHA-256 checksums in the model repository allow any reader to verify which files are loaded [29].